Open in Colab

Prelab 10.1 Machine Learning 3 - Real-time Sound Recognition for Classification

Problem definition and understanding data flow

Learning Goals

Studetns will be able to:

1. Understand sound data format and signal processing
2. Labeling classification for machine learning model
3. Training CNN (Convolutional Neural Network) model for sound recognition

1.1 Introduction

In Lab 10, as the last lab activity, we will implement real-time machine monitoring using sound recognition. Machine sound monitoring is extensively utilized for various applications such as operational state, prognostic and diagnostic monitoring. Because machine-emitted sound contains the operational process information, humans can tell differences based on audible sounds from the machine. Moreover, in the medical industry, a doctor examines a patient using a stethoscope to listen to the sound of organs. Based on the sound they learned and trained, listening to the sound, doctors determine if the patient is healthy. Inspired by this human ability and examining method, Purdue LAMM (Laboratory for Advanced Multiscale Manufacturing) developed an internal sound sensor and monitoring system as Figure 1. It was also introduced in Purdue Research Foundation News.

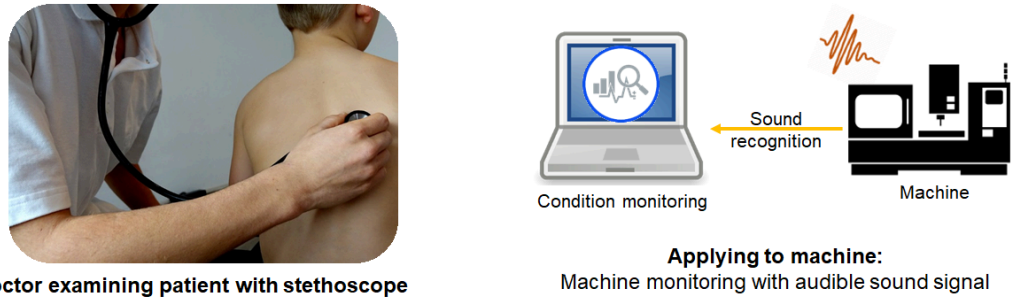


Figure 1 Sound recognition applications for humans and machines

Let's think about the auditory pathway of human beings and machine sound monitoring. In the sense of monitoring, ears are sound sensors that can capture pressure/density fluctuations of a medium, normally air. To be specific, the eardrum receives sound signals and the malleus amplifies the signals and transmits them to the cochlea. The cochlea then filters the signals to a logarithmic scale. The filtered and compressed information are received by the auditory cortex and which processes auditory information. Lexicons from knowledge and experience are in the hippocampus. Lastly, based on the process of the hippocampus, the frontal lobe recognizes the sound signals and then executes actions if needed. These proccesses can be applied to machine sound recognition in the same manner. Figure 2 illustrates analogy between auditory pathway of human and machine sound monitoring.

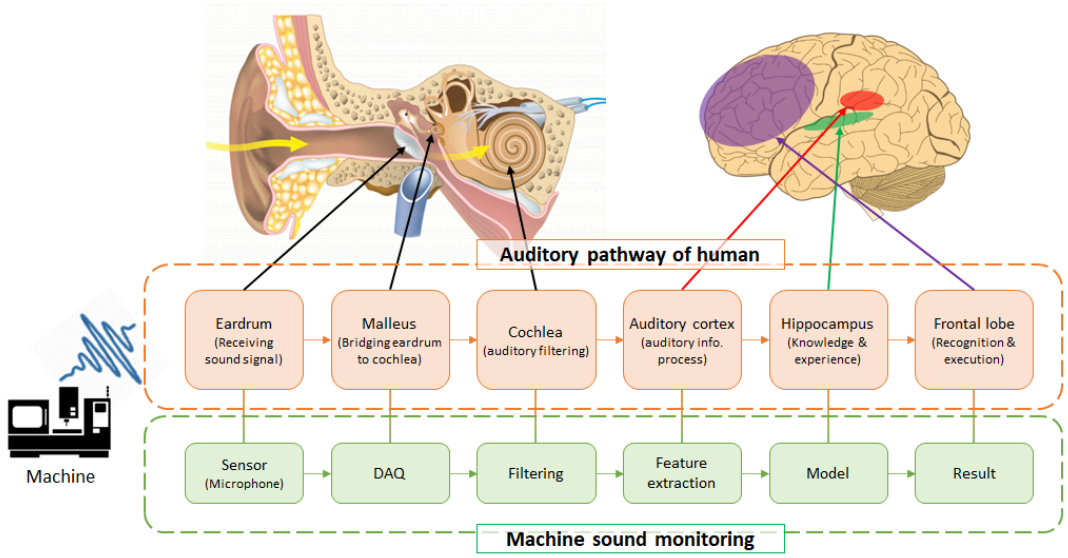


Figure 2 Auditory pathways for sound recognition by human and machine monitoring

In Lab10 as a hands-on activity, we will perform sound recognition using the internal sound sensor to predict the operational states of a vacuum pump. In this prelab, we will go through 1) sound data by experiments and understanding sound stream by MTConnect, 2) data visualization to create a training dataset, 3) CNN (convolutional neural network) model training for classification, and 4) install relevant Python packages for sound signal processing.

1.2 Sound sensor and target machine

The internal sound sensor consists of a stethoscope and a USB microphone. By attaching the microphone at the end of the rubber side of the stethoscope, the sensor captures sounds from the head of the stethoscope. Figure 3 shows configurations of the internal sound sensor. It allows us to collect not airborne sound but structure-borne sound. In acoustic expression, it is able to capture the near-field sound effect of the structure. The diaphragm plays a role in reducing high-frequency components. Whereas, the bell amplifies the low-frequency components (10 - 200 Hz). Therefore, the sound signals from the sensor are some what different from the airborne sound.

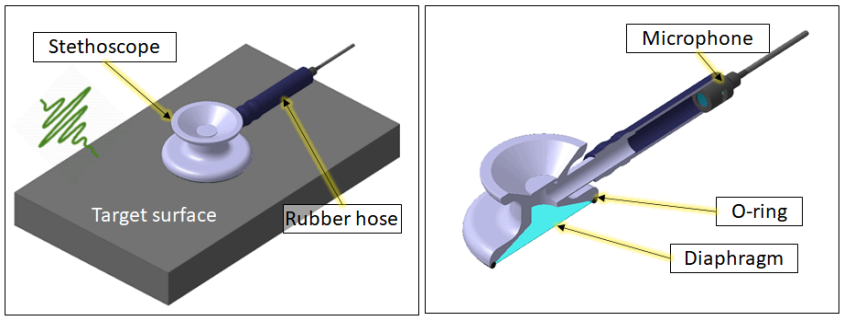


Figure 3 configuration of the internal sound sensor

The target machine is a vacuum pump for degassing system which we mainly used in Lab 3 and Prelab 8.2. The internal sound sensor was installed on the vacuum pump as Figure 4. Note that the small bell side without a diaphragm was placed on the target surface in this case because of limited space.

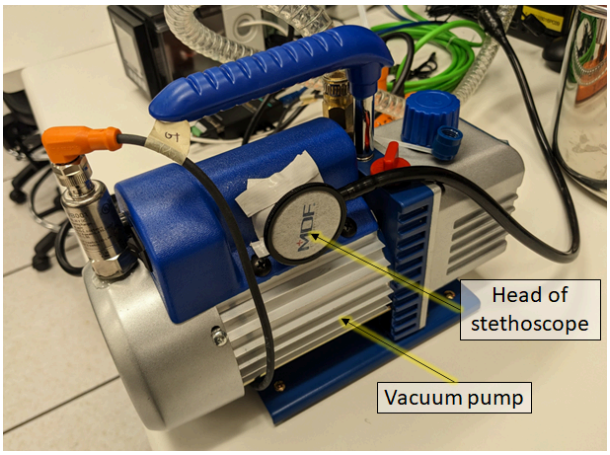


Figure 4 Vacuum pump and sound sensor

1.3 Monitoring system and sound stream flow

The monitoring system and data stream of the sound signals using MTConnect are illustrated in Figure 5. The stethoscope captures sound from the target surface of the machine. The USB microphone collects the analog sound signals and then convert them to the digital signals by ADC (analog-to-digital converter). The MTConnect adapter in the headless server computer samples and converts the digital sound signals (byte arrays, 2^n number of sample points) to the space delimited 16-bit signed Int arrays with the timestamp. Finally, the sound stream is accessible from the MTConnect agent of the server. By observing MTConnect standard, *DisplacementTimeSeries* data type was used. It contains sound data in **48 kHz sampling rate with 2^{11} chunk size and 16-bit depth resolution of the mono-channel**. In other words, each sample (sequence in MTConnect) contains 2^{11} (= 2048, chunk size) sound data points. The timestamp of MTconnect is the exact time of the last observation. Therefore, the timestamp of each sound measurement is tracible. Based on this information, perform Task 1.1 below.

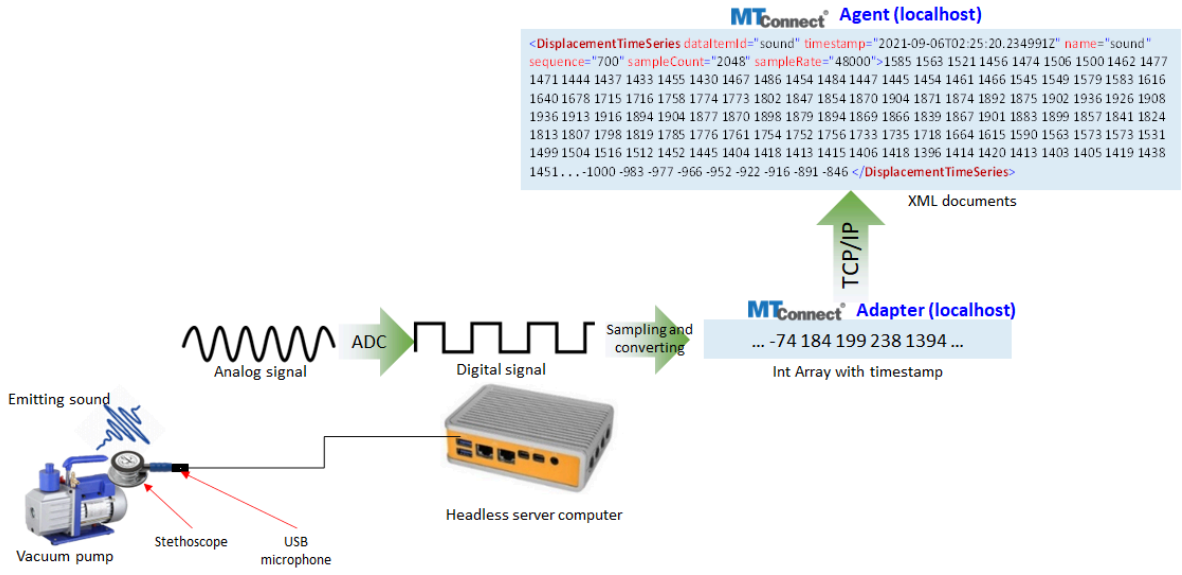


Figure 5 Vacuum pump and sound sensor

Task 1.1

With sound (audio) sensing specifications are described above. Answer the following questions.

1. How long does each sample (sequence) in MTConnect stand for? Answer this in the second unit.

- Hint: Sampling rate is 48 kHz and each sequence has 2048 sample points.

If the sampling rate is 48 kHz and each sequence has 2048 sample points, then each sample sequence is:

$$T = \frac{No.\ Sample\ Points}{Frequency} = \frac{2048\ points}{48 \times 10^3 s^-1} = 0.04267s$$

---

2. If you want to get approximately 1 second-long sound signals, how many consequent sequences do you need from the MTConnect agent?

To get approximately 1 second-long sound signals, we would need

$$\frac{1s}{T} = \frac{1s}{0.04267s} = 23.4375\ consequent\ sequences$$

1.4 Problem definition and classification

As we did in the previous lab (Prelab8), we will create and implement a machine learning model to predict the operational states of the vacuum pump. We will set the problem as muti-class classification, whereas the previous lab defined it as anomaly detection using an autoencoder architecture. In other words, in the previous lab, we assumed that the vacuuming is normal and the air leakage is abnormal. It is a straightforward approach to defining an anomaly prediction problem. However, we have another operational state, which is the machine is off. As you saw in Lab9, when the machine is off, the autoencoder model predicts it is abnormal based on the MAE (mean absolute error) threshold. Let's convert the problem definition from anomaly detection to classification. Including the vacuuming and air-leaking states, normally, the vacuum pump has three states (classes) as below. It is illustrated in Figure 6.

- Class 1 (state 1): OFF
- Class 2 (state 2): ON, Vacuumming
- Class 3 (state 3): ON, Air-leaking

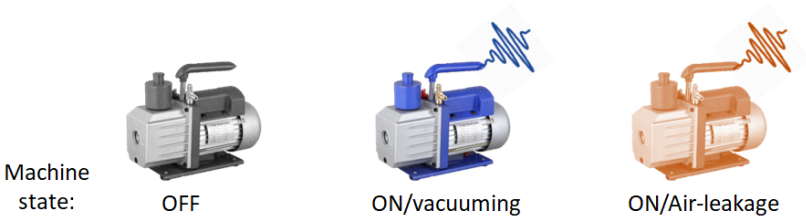


Figure 5 Operational states for vacuum pump

Of course, there are many different ways to define the problem including the machine learning model configurations. For example, you may have a combined algorithm from two machine learning models; one is for an operational state (ON/OFF) prediction model and another is for an anomaly detection model (Normal (vacuuming)/Abnormal (air-leaking)). It is able to perform the same prediction/monitoring task for the vacuum pump sound recognition. Yet, we will deal with a multi-class classification using CNN (convolutional neural network). CNN will be described in detail in the following sessions.

Let's play two video clips below to figure out airborne sound differences between the states. Then perform Task 1.2.

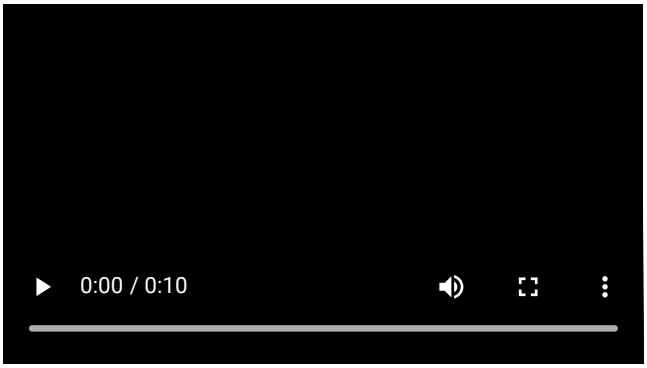
Video clip 1: ON/vacuuming state

In [ ]:
```
# Don't run this code block.
# Just play the vide clip on the output cell below.
```

```
from IPython.display import HTML
from base64 import b64encode
mp4 = open('Prelab10_Video1_Air-tight_compressed.mp4','rb').read()
data_url = "data:video/mp4;base64," + b64encode(mp4).decode()

HTML("""
<video width=400 controls>
      <source src="%s" type="video/mp4">
</video>
""" % data_url)
```

Out[ ]:



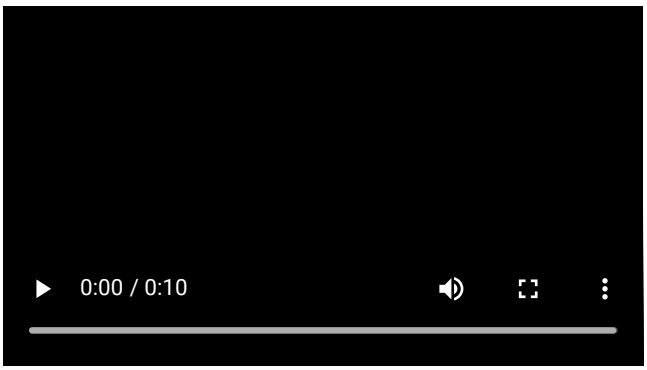
Video clip 2: ON/air-leaking state

You see the vacuum gauge and the cover is not closed in the video.

In [ ]:
```
# Don't run this code block.
# Just play the vide clip on the output cell below.
from IPython.display import HTML
from base64 import b64encode
mp4 = open('Prelab10_Video2_Air-leakage_compressed.mp4','rb').read()
data_url = "data:video/mp4;base64," + b64encode(mp4).decode()

HTML("""
<video width=400 controls>
      <source src="%s" type="video/mp4">
</video>
""" % data_url)
```

Out[ ]:



Task 1.2

Do you think you can determine the operational states between vacuuming and air-leaking only using sound? Describe the differences in terms of sound.

Yes, I believe I can determine the operational states between vacuuming and air-leaking only using sound. The differences between the vacuuming state and air-leak state is that the air-leak state has a lower pitch and is louder comparatively.

In the following session, we will go through sound signal processing and labeling for CNN model training.

Please continue to Prelab 10.2 here.